Miscellaneous (Variable, Block, Constructor, this keyword):-

- Scope of Local variable starts from its declaration statement and it goes up to end of the block where it is declared.
- Local variables must be accessed from or after declaration statement.
- Class level variable can be declared anywhere in the class. It can be accessed from other members.
- You can access class level variable from methods, constructors directly.
- When static variables declared after static initialization block then remember following points.
    a) You can initialize the variable directly.
    b) You cannot access value of the variable directly.
    c) When you access value of variable directly, compiler will generate error called "illegal forward reference".
    d) You can access the value of the variable with class name.
- When instance variables declared after instance initialization block then remember following points.
    a) You can initialize the variable directly.
    b) You cannot access value of the variable directly.
    c) When you access value of variable directly, compiler will generate error called "illegal forward reference".
    d) You can access the value of the variable with class name.
- When you are declare instance variable and initialization block then initialization of the variable and execution of the initialization blocks will be in the same order as defined in the class.
- Final instance variable can be initialized from the constructor also but it must be initialized from all the constructors if available.
- When final instance variable initialized from the instance initialization blocks then that variable cannot be initialized from constructor.
- Final static variable can be initialized from the static initialization blocks only; it cannot be initialized from constructors and instance initialization blocks.
- Class loading is the process of reading the required [.class] file and storing that information in the memory.
- Class will be loaded only once in the memory by the JVM. After loading the class if you delete the class file then also the application will be executed.
- The class will be loaded in the memory by the JVM when first time the member of the class will be used by JVM.
    a. When you are executing the class with java.exe then class will be loaded and static block will be executed.

class Hello	class Hello {
	static {
	System.out.println("Static block");
	}
	public static void main(String str[]){
	}
	}

Note:- From java 7 if you don't have main method then class won't be loaded and static block won't be executed.

b. When you are creating object of the class and before this statement no other member of the class is used then class will be loaded.

```
class Exe {
public static void main(String str[]){
new Hello ();
new Hello ();
  }
}
```

c. When you access the members of the class and before this statement no other member of the class is used then class will be loaded.

```
class Exe1 {
public static void main(String str[]){
System.out.println(Hello.a);
  }
}
```

d. When you access the final static variables that are initialized in the statement then JVM won't execute the static block.

```
class Exe1 {
public static void main(String str[]){
System.out.println(Hello.a);
  }
}
class Hello {
final static int a=10;
static {
System.out.println("Static Block");
  }
}
```

e. When you define the reference variable but not using any members of the class then class will not be loaded.

```
class Exe {
public static void main(String str[]){
Hello h=null;
  }
}
```

- Final variable with static having default value when class name loaded.
- Final variable with Instance having default value when this will be used.

class Hello {
final static int a;
static {
  System.out.println (Hello. a);
}
	class Hello {
	final int a;
	static {
	  System.out.println (this. a);
	}

- If inside your class you have Instance block and Constructor than while creating the class file the statement of instance block will be copied to the Constructor.

Hello.java file	**Hello. class** file

class Hello {
Hello(){
System.out.println("Def cons");
}
Hello(int a){
System.out.println("1-Arg cons");
}
{
System.out.println("Ins Block");
}
}
}
	class Hello {
	Hello(){
	System.out.println("Ins Block");
	System.out.println("Def cons");
	}
	Hello(int a){
	System.out.println("Ins Block");
	System.out.println("1-Arg cons");
	}
	}

- If you are using this keyword to invoked overloaded constructor than instance block will not be copied in the Constructor.

Hello.java	**Hello. class**

class Hello {
Hello(){
System.out.println("Def cons");
}
Hello(int a){
this();
System.out.println("1-Arg cons");
}
{
System.out.println("Ins Block");
}
}
}
	class Hello {
	Hello(){
	System.out.println("Ins Block");
	System.out.println("Def cons");
	}
	Hello(int a){
	this();
	System.out.println("1-Arg cons");
	}
	}

Question: Consider the following program and write the order of execution.

Hello1.java	**step of execution:**

class Hello {
static int a=10;
static{
System.out.println ("SB1: "+Hello.a);
}
static{
System.out.println ("SB2: "+Hello.a);
}
}
	1. Memory for static variable will be allocated.
	2. Static variable will be initialized with default value.
	3. Static variable will be initialized with new value.
	4. Static block 1 will be executed
	5. Static block 2 will be executed.

Hello1.java	**step of execution:**

class Hello {
static{
System.out.println("SB1: "+Hello.a);
}
static int a=10;
static{
System.out.println("SB2: "+Hello.a);
}
}
	1. Memory for static variable will be allocated.
	2. Static variable will be initialized with default value.
	3. Static variable will be initialized with default value.
	4. Static block 1 will be executed
	5. Static variable will be initialized with new value.
	6. Static block 2 will be executed.

Hello3.java	**step of execution:**

class Hello {
static{
System.out.println ("SB1: "+Hello.a);
}
static{
System.out.println ("SB2: "+Hello.a);
}
static int a=10;
}
	1. Memory for static variable will be allocated.
	2. Static variable will be initialized with default value.
	3. Static variable will be initialized with default value.
	4. Static block 1 will be executed.
	5. Static block 2 will be executed.
	6. Static variable will be initialized with new value.

Programs:

OOPS94.java	**OOPS95.java**

class OOPS94{
public static void main(String args[]){
System.*out*.println(a);
int a=10;
}
static{
System.*out*.println(b);
int b=20;
}
}
	class OOPS95{
	public static void main(String args[]){
	System.*out*.println(*b*);
	}
	static int *b*=20;
	}

OOPS96.java	**OOPS97.java**

class OOPS96{
public static void main(String args[]){
System.*out*.println(Hello. *b*);
}
}
class Hello{
static{
b=10;
}
static int *b*=90;
}
	class OOPS97{
	public static void main(String args[]){
	System.*out*.println("Main : "+Hello. *b*);
	}
	}
	class Hello{
	static{
	Hello. *b*=10;
	}
	static int *b*=90;
	}

OOPS98.java

```
class OOPS98{
public static void main(String args[]){
System.out.println ("Main: "+Hello. b);
}}
class Hello{
static{
    System.out.println (b);
  }
static int b=20;
}
```

OOPS99.java

```
class OOPS99{
public static void main(String args[]){
System.out.println ("Main : "+Hello. b);
}}
class Hello{
static{
   System.out.println ("SB :  "+Hello. b);
 }
static int b=20;
}
```

OOPS100.java

```
class OOPS100{
public static void main(String args[]){
System.out.println ("Main: "+Hello. b);
}}
class Hello{
static{
  System.out.println ("SB 1: "+Hello. );
 }
static int b=20;
static{
    System.out.println ("SB2: "+b);
  }
}
```

OOPS101.java

```
class OOPS101{
public static void main(String args[]){
System.out.println ("Main: "+Hello. b);
}}
class Hello{
static{
   System.out.println ("SB 1: "+Hello. b);
 }
static int b=20;
 static{
     System.out.println ("SB 2: "+Hello. b);
   }
}
```

OOPS102.java

```
class OOPS102{
public static void main(String args[]){
System.out.println ("Main : "+Hello. b);
  }
}
class Hello{
static int a=20;
  static{
    System.out.println("SB1 "+a);
    System.out.println("SB1 "+b);
  }
static int b=20;
static{
   System.out.println("SB2 "+a);
   System.out.println("SB2 "+b);
}}
```

OOPS103.java

```
class OOPS103{
public static void main(String args[]){
System.out.println ("Main : "+Hello. b);
  }
}
class Hello{
static int a=10;
static{
  System.out.println ("SB1 "+a);
  System.out.println ("SB1 "+Hello. b);
 }
static int b=20;
static{
  System.out.println("SB2 "+a);
  System.out.println("SB2 "+b);
}}
```

OOPS104.java

```
class OOPS104{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
}}
```

```
class Hello{
 {
  a=90;
 }
int a=20;
}
```

OOPS105.java

```
class OOPS105{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
 {
  this. a=90;
  }
int a=20;
}
```

OOPS106.java

```
class OOPS106{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
 {
  System.out.println(a);
 }
int a=20;
}
```

OOPS107.java

```
class OOPS107{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
  {
   System.out.println(this. a);
  }
int a=20;
}
```

OOPS108.java

```
class OOPS108{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
  {
   System.out.println ("IB1 "+this. a);
  }
int a=20;
 {
   System.out.println("IB1 "+a);
  }
}
```

OOPS109.java

```
class OOPS109{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
 {
   System.out.println ("IB1 "+this. a);
 }
int a=20;
 {
  System.out.println ("IB1 "+this. a);
 }
}
```

OOPS110.java

```
class OOPS110{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
int a=10;
  {
   System.out.println("IB1 "+a);
   System.out.println("IB1 "+b);
  }
int b=20;
 {
    System.out.println("IB1 "+a);
    System.out.println("IB1 "+b);
  }
}
```

OOPS111.java

```
class OOPS111{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
int a=10;
 {
   System.out.println("IB1:  "+a);
   System.out.println("IB1:  "+this.b);
 }
int b=20;
 {
  System.out.println("IB1: "+a);
  System.out.println("IB1: "+b);
 }
}
```

OOPS112.java

```
class OOPS112{
public static void main(String args[]){

Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
Hello()
 {
 System.out.println("con : "+a);
 }
int a=10;
}
```

OOPS113.java

```
class OOPS113{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
 {
   System.out.println ("IB : "+this. a);
 }
Hello(){
   System.out.println("Cons : "+a);
}
int a=10;
}
```

OOPS114.java

```
class OOPS114{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
 int a=10;
static  {
    String a="JLC";
    System.out.println("IB : "+a+"\t"+a);
  }
}
```

OOPS115.java

```
class OOPS115{
public static void main(String args[]){
Hello h=new Hello();
System.out.println("Main : "+h.a);
  }
}
class Hello{
int a=10;
static  {
 String a="India";
  Hello h=new Hello();
  System.out.println("IB : "+a+"\t"+h.a);
  }
}
```

OOPS116.java

```
class OOPS116{
public static void main(String args[]){
System.out.println("Main : "+Hello.a);
  }
}
class Hello{
static int a=10;
 {
  System.out.println("IB : "+this);
  Hello h=new Hello();
 }
}
```

OOPS117.java

```
class OOPS117{
public static void main(String args[]){
System.out.println("Main : "+Hello.a);
new Hello();
  }
}
class Hello{
static int a=10;
 {
  System.out.println("IB : "+this);
  Hello h=new Hello();
 }
}
```

OOPS118.java

```
class OOPS118{
public static void main(String args[]){
System.out.println("Main : "+Hello.a);
}
}
class Hello{
static int a=10;
 {
   System.out.println("IB : ");
 }
static{
  System.out.println("SB : ");
 }
static Hello h=new Hello();
}
```

OOPS119.java

```
class OOPS119{
public static void main(String args[]){
System.out.println("Main : "+Hello.a);
}
}
class Hello{
static int a=10;
static Hello h=new Hello();
  {
     System.out.println("IB  ");
  }
static{
    System.out.println("SB ");
  }
}
```

OOPS120.java

```
class OOPS120{
public static void main(String args[]){
System.out.println("Main : "+Hello.a);
}
}
class Hello{
static int a=10;
static Hello h1=new Hello();
 {
   System.out.println("IB  ");
 }
static{
   System.out.println("SB  ");
  }
static Hello h2=new Hello();
}
```

OOPS121.java

```
class OOPS121{
public static void main(String args[]){
System.out.println("Main : "+Hello.a);
}}
class Hello{
static int a=10;
static Hello h1;
 {
  System.out.println("IB  ");
 }
static{
   System.out.println("SB  ");
  }
static Hello h2;
}
```

OOPS122.java

```
class OOPS122{
public static void main(String args[]){
new Hello();
}}
class Hello{
int a;
Hello h1;
 {
   System.out.println("IB ");
 }
static{
   System.out.println("SB ");
 }
}
```

OOPS123.java

```
class OOPS123{
public static void main(String args[]){
new Hello();
  }
}
class Hello{
int a=10;
Hello h1= new Hello();
 {
  System.out.println("IB ");
 }
static{
  System.out.println("SB ");
 }}
```

OOPS124.java

```
class OOPS124{
public static void main(String args[]){
new Hello();
  }
}
class Hello{
 {
    System.out.println("IB  ");
 }
Hello() {
   System.out.println("DC ");
   new Hello();
 }
}
```

OOPS125.java

```
class OOPS125{
public static void main(String args[]){
new Hello();
 }
}
class Hello{
 {
   System.out.println("IB ");
 }
Hello(){
  new Hello();
  System.out.println("DC ");
 }
}
```

OOPS126.java

```
class OOPS126{
public static void main(String args[]){
new Hello();
  }
}
class Hello{
 {
    System.out.println("IB ");
 }
Hello(){
  System.out.println("DC ");
 }
Hello(int a){
  System.out.println("1-Arg Con");
 }
}
```

OOPS127.java

```
class OOPS127 {
public static void main(String args[]){
new Hello();
  }
}
class Hello{
  {
    System.out.println("IB ");
  }
Hello(){
   System.out.println("DC ");
  }
Hello(int a){
  this();
     System.out.println("1-Arg Cons");
  }
}
```

OOPS128.java

```
class OOPS128{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
 }
}
 class Hello{
    final int a;
}
```

OOPS129.java

```
class OOPS129{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
  }
}
class Hello{
 final int a;
  {
  a=10;
   }
}
```

OOPS130.java

```
class OOPS130{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
 }
}
class Hello{
final int a;
Hello(){
  a=10;
  }
}
```

OOPS131.java

```
class OOPS131{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
  }
}
class Hello{
final int a;
Hello(){
   a=10;
}
Hello(int a)  { }
}
```

OOPS132.java

```
class OOPS132{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
}}
class Hello{
final int a;
Hello() {
    a=10;
}
Hello(int a){
  this. a=a;
}}
```

OOPS133.java

```
class OOPS133{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
  }}
class Hello{
 final int a;
Hello(){
  a=10;
 }
 {
  a=90;
 }
}
```

OOPS134.java

```
class OOPS134{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
  }
}
class Hello{
static final int a;
{
  a=90;
  }
}
```

OOPS135.java

```
class OOPS135{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
 }
}
class Hello{
static final int a;
Hello(){
a=90;
  }
```

OOPS136.java

```
class OOPS136{
public static void main(String args[]){
Hello h=new Hello();
System.out.println(h.a);
  }
 }
class Hello{
static final int a;
static{
     System.out.println("SB"+a);
  }
}
```

OOPS137.java

```
class OOPS137{
public static void main(String args[]){
System.out.println(Hello.a);
  }
}
class Hello {
static final int a;
static {
 System.out.println("SB: "+Hello.a);
 a=10;
 }
}
```

OOPS138.java

```
class OOPS138{
public static void main(String args[]){
System.out.println(Hello.a);
  }
}
class Hello{
static final int a=10;
static{
   System.out.println("SB");
  }
}
```

OOPS139.java

```
class OOPS139{
public static void main(String args[]){
System.out.println(Hello.a);
}}
class Hello{
static final int a;
static{
a=10;
System.out.println("SB");
  }
}
```

OOPS140.java

```
import java. util.*;
class OOPS140 {
public static void main(String args[]){
System.out.println (Hello.a);
System.out.println ("Delete class file and
press Enter");
Scanner sc= new Scanner(System. in);
sc. nextLine();
Hello h1= new Hello();
h1.show();
Hello h2= new Hello();
h2.show();
  }
}
class Hello{
static int a=10;
Hello() {
  System.out.println("** Hello Def
const**");
  }
Hello(int a) {
  System.out.println(" ** Hello(int)
const**");
  }
void show() {
    System.out.println("** show()**");
  }
}
```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS94	Compilation error	[Error: cannot find symbol b] because, b must have to initialize before call.
OOPS95	20	step of execution 6. Memory for static variable will be allocated. 7. Static variable will be initialized with default value. 8. Static variable will be initialized with new value.
OOPS96	90	Same as 625, but value of variable b=10 will removed because it is not in use. for reference (d compiler)
OOPS97	Main : 90	Same as 625, but value of variable b=10 will removed because it is not in use. for reference (d compiler)
OOPS98	Compilation error	[Error: illegal forward reference] Because, variable is declared after used.
OOPS99	SB : 0 Main : 20	1st memory for static variable will allocated then static variable will be initialized with default value then initialized with new value.
OOPS100	SB1 : 0 SB2 : 20 Main : 20	1st memory for static variable will allocated then static variable will be initialized with default value then initialized with new value.
OOPS101	SB 1 : 0 SB 2 : 20 Main : 20	1st memory for static variable will allocated then static variable will be initialized with default value then initialized with new value.
OOPS102	Compilation error	[Error: illegal forward reference] Because, variable "b" is declared after used.
OOPS103	SB1: 10 SB1: 0 SB2: 10 SB2 :20 Main : 20	To access static variable in static block use class name as a reference if variable is declared after use. If before use that variable is declare in class than no need to use class name to access that variable.
OOPS104	Main : 20	1st class is loaded than object will created then memory will be allocated for instance variable with default value then initialize with new value.
OOPS105	Main : 20	same as 534
OOPS106	Compilation error	[Error: illegal forward reference] Because, variable "a" is declared after used.
OOPS107	0 Main : 20	To access class level variable use this keyword as reference but it must be initialize before use.
OOPS108	IB1: 0 IB1: 20 Main : 20	To access class level variable use this keyword as reference but it must be initialize before use.
OOPS109	IB1: 0 IB1: 20 Main : 20	To access class level variable use this keyword as reference but it must be initialize before use.

Prog No.	Output	Error and Details
OOPS110	Compilation error	[Error: Cannot reference a field before it is defined]
OOPS111	IB1: 10 IB1: 0 IB1: 10 IB1: 20 Main : 10	For overcome previous problem must use this keyword.
OOPS112	Cons : 10 Main : 10	1st memory allocated then initializes that variable than constructor will be executed then main method statement.
OOPS113	IB : 0 Cons : 10 Main : 10	1st memory allocated then initialize variable with default value than instance block then initialize with new value then constructor will be executed then main method statement.
OOPS114	IB : India India Main : 10	Variable "a" is declared before used.
OOPS115	IB : India 10 Main : 10	Here, we are creating a object to refer the value of instance variable.
OOPS116	Main : 10	Here, Instance block is executed due to object creation and this keyword is use to refer the variable.
OOPS117	IB : Hello@70610a IB : Hello@1f31652 IB : Hello@3e96cf …… …… StackOverflowError	Here every time object will created and memory allocated and deal located so that after some time it will give stack Over Flow Error.
OOPS118	SB IB Main : 10	Here, First memory allocated then initialize the variable with default value then new value then static block will be executed then instance block then statement of main method.
OOPS119	IB SB Main : 10	Same as 648 but we are creating static object so instance block will execute first then static block.
OOPS120	IB SB IB Main : 10	Same as 648 but we are creating static object so instance block will execute first then static block and we are creating static object so again IB will executed.
OOPS121	SB Main : 10	Here, instance block will not execute because object is not creating.
OOPS122	SB IB	1st static block then instance block will execute.
OOPS123	SB StackOverflowError	Here, 1st static block will executed then give error because object will creation.

Prog No.	Output	Error and Details
OOPS124	IB DC IB DC …… StackOverflowError	same ”
OOPS125	IB IB …… StackOverflowError	same ”
OOPS126	IB DC	We are not passing any argument on the time of object creation.
OOPS127	IB DC	same”
OOPS128	Compilation error	[Error: The blank final field a may not have been initialized]
OOPS129	10	Here, Object created so instance block will executed so that value will initialize.
OOPS130	10	same”
OOPS131	Compilation error	[Error: variable a might not have been initialized]
OOPS132	10	Here, value is called by this. a is valid .
OOPS133	Compilation error	[Error: variable a might not have been initialized] we cannot assign value of variable in two place.
OOPS134	Compilation error	[Error: cannot assign a value to final variable a]
OOPS135	Compilation error	[Error: cannot assign a value to final variable a]
OOPS136	Compilation error	[Error: The blank final field a may not have been initialized and The final field Hello.a cannot be assigned]
OOPS137	SB: 0 10	static final variable will assigned in static block
OOPS138	10	Hello.a is replaced by value because it is static final variable.
OOPS139	SB 10	Same as 667
OOPS140	10 **Delete class file and press Enter** ** Hello Def const ** show() ** Hello Def const ** show()	Here, we are verifying that if “.class” file will delete after compilation it will executed or not..... it is executed successfully due to Class will be loaded only once in the memory by the JVM. After loading the class if you delete the class file then also the application will be executed.